Car Rental Application

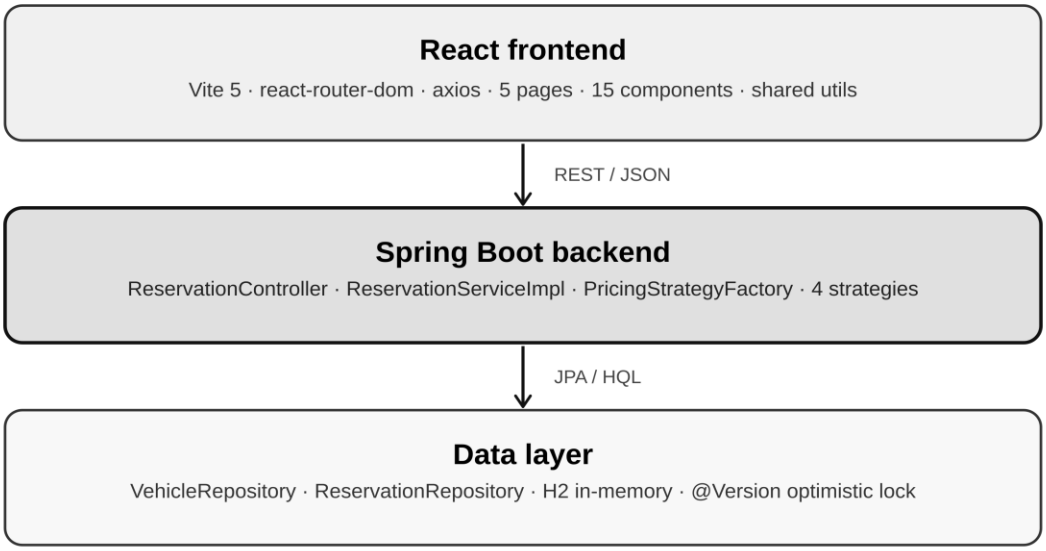
Technical Design Document

---

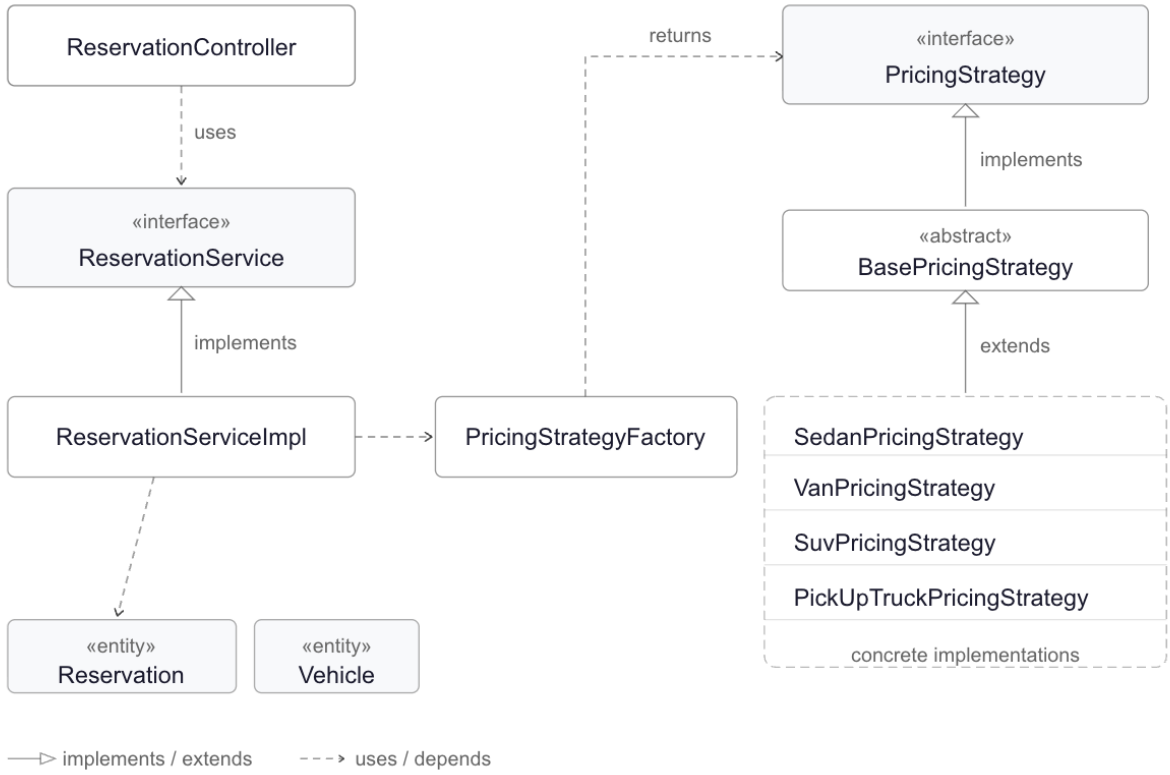
1. Architecture Overview

The Car Rental Application is built as a full-stack system with a Java Spring Boot backend and a React frontend. The architecture follows a layered approach separating presentation, business logic, and data concerns.

Backend	Java 21, Spring Boot 3, Spring Data JPA, H2 (in-memory), Rest Api
Frontend	React 18, Vite 5, react-router-dom, axios
Database	H2 in-memory — seeded via DataLoader
Build	Gradle (backend), npm (frontend)



Frontend → HTTP → Controller → Service → Strategy → Repository → H2



2. Performance Optimizations — Frontend (React)

Five distinct performance techniques were applied to the React frontend. Each technique targets a specific inefficiency in rendering or bundle loading.

2.1 React.memo

Prevent child components re-rendering when parent state changes

Problem	Every time ResultsPage state changes, all 4 VehicleRow components re-render even when their props are identical.
Solution	Wrap components in React.memo. React shallowly compares props — if nothing changed, the component is skipped entirely.
Applied to	CarIcon, CategoryBadge, VehicleRow, PricingSummary
Benefit	Fewer renders, smoother UI, lower CPU usage on state changes

2.2 useCallback

Stable function references so memoed children are not broken

Problem	Every render creates a new handleBookNow function. VehicleRow receives a new prop reference — React.memo is bypassed.
Solution	useCallback returns the same function instance across renders. Dependencies array controls when a new function is created.
Applied to	handleBookNow, getCheapestCategory in ResultsPage; handleChangeVehicle in BookingPage
Benefit	Memo stays effective, prevents wasted renders

2.3 Lazy Loading + Suspense

Split JS bundle — load each page only when user navigates to it

Problem	Without lazy loading, all 5 pages download on first visit — including ModifyPage and CancelPage the user may never need.
Solution	Vite creates separate JS chunks per page. Each chunk is fetched only when the user navigates there.
Applied to	All 5 pages in App.jsx — HomePage, ResultsPage, BookingPage, ModifyReservationPage, CancelReservationPage
Benefit	Smaller initial bundle, faster first load time

2.4 Correct List Keys

Stable unique keys let React identify items and avoid unnecessary unmounts

Problem	Using array index as key means React cannot track which item moved when the list re-sorts by price.
Solution	Keys from stable data (vehicleCategory, row.label) let React reuse existing DOM nodes.
Applied to	options.map in ResultsPage uses vehicleCategory; rows.map in AmountBreakdown uses row.label
Benefit	Fewer DOM operations, no flicker on re-sort

2.5 useTransition

Mark non-urgent updates so React keeps UI responsive during heavy renders

Problem	When API responds with vehicle options, React synchronously renders all 4 rows. The loading spinner can freeze before disappearing.
Solution	startTransition marks setOptions as low priority. React removes the spinner first, then renders rows.
Applied to	setOptions inside fetchOptions in ResultsPage
Benefit	UI stays responsive during heavy render, prioritised update order

3. Performance Optimizations — Backend (Java)

Nine performance techniques were applied to the Java backend, targeting memory allocation, CPU efficiency, and logging overhead.

Technique	Where Applied	Benefit
Java record for PricingRequest	PricingRequest.java	Immutable, no builder overhead, auto methods
Java record for BreakdownLineDto	BreakdownLineDto.java	Read-only DTO, faster Jackson serialisation
EnumMap in PricingStrategyFactory	PricingStrategyFactory.java	Array-backed O(1) lookup, no hashing
static final Logger	All strategy classes + service	One logger per class, no GC pressure
isDebugEnabled() guard	All strategy classes + service	Prevents string concatenation when debug off
List.of() for breakdown	Sedan, Van, SUV strategies	Immutable compact array-backed list
ArrayList(3) pre-sized	PickupTruck strategy	Avoids internal array resize and copy
Primitive double/int constants	All strategy classes	No boxing, values stay on stack

Strategy fetched once per call	ReservationServiceImpl	Single EnumMap lookup not two per request
get(0) not stream	Vehicle lookup in service	No stream overhead for first element pick
Vehicle extracted to local var	ReservationServiceUtil mapper	Avoids repeated lazy proxy calls

Records as Lightweight Data Carriers

PricingRequest and BreakdownLineDto were converted from @Data classes to Java records. Records are immutable, generate equals/hashCode/toString automatically, and are optimised by the JVM more aggressively than regular classes.

EnumMap for O(1) Strategy Lookup

The PricingStrategyFactory uses EnumMap instead of HashMap. EnumMap uses a plain array internally indexed by enum ordinal — no hashing, lower memory, and guaranteed O(1) access.

---

4. OOP Principles and Design Patterns Applied

Six OOP principles and patterns were applied across the backend codebase. Together they eliminate code duplication, enforce single responsibility, and make the system easy to extend.

Pattern	Where	What it solves
Strategy	PricingStrategy + 4 classes	Swap pricing algorithm per vehicle type

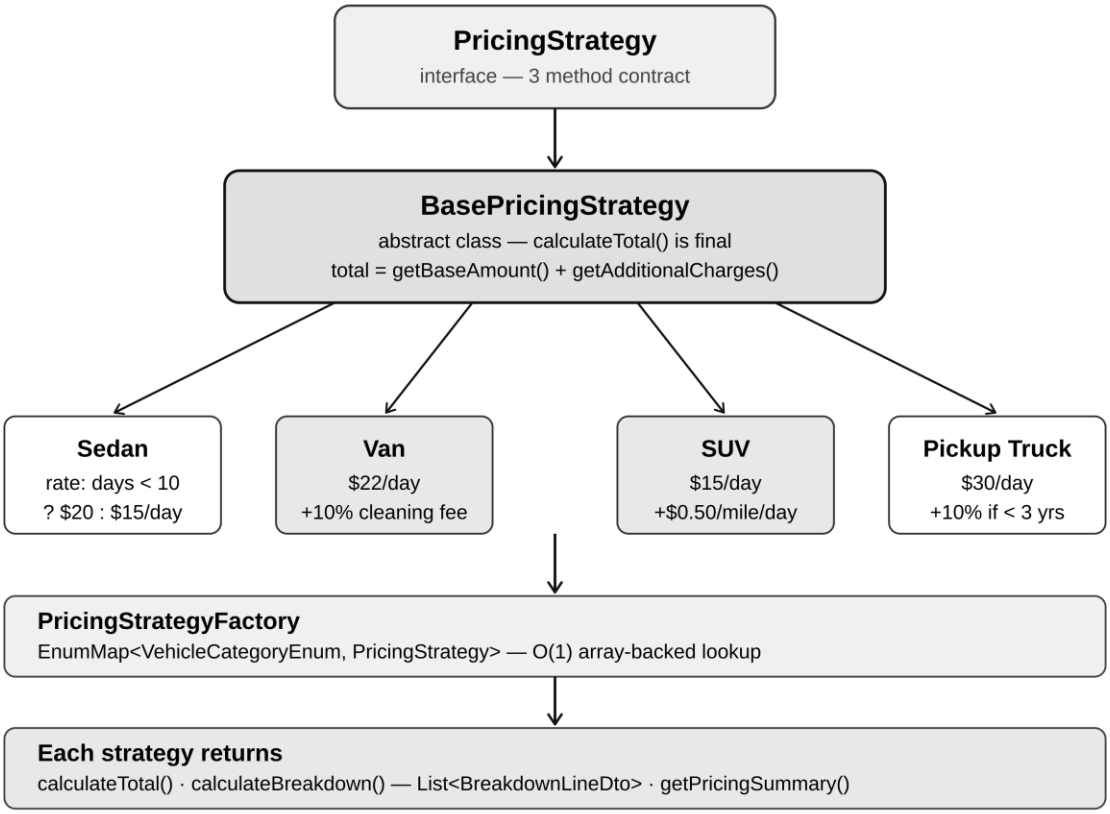
Factory	PricingStrategyFactory	Hide which strategy concrete class is used
Repository	VehicleRepository, ReservationRepository	Decouple persistence from business logic
DTO	ReservationRequestDto, ReservationResponseDto	Decouple API shape from entity shape
Builder	@Builder on entities and DTOs	Readable object construction
Singleton	All Spring beans	One shared instance, no repeated init cost
Facade	ReservationService interface	Simple surface hiding complex orchestration
Observer	@PrePersist on Reservation	Auto-set fields on lifecycle events

4.1 Abstraction — PricingStrategy Interface

The PricingStrategy interface defines a contract that all vehicle pricing implementations must satisfy. The service layer depends only on this interface — it never knows which concrete strategy it is calling.

4.2 Strategy Pattern — PricingStrategyFactory

The PricingStrategyFactory holds an EnumMap of all strategies. The service calls getStrategy(category) and receives the correct PricingStrategy at runtime. No if/else or switch on vehicle type in the service.



4.3 Encapsulation — Entity Domain Methods

Vehicle and Reservation entities expose behaviour methods rather than raw setters. This means the service tells entities what to do — it does not manipulate fields directly.

4.4 Single Responsibility Principle — Private Helpers in Service

ReservationServiceImpl was refactored to extract each concern into a named private method. Each public method (reserveCar, modifyReservation, etc.) reads as a high-level orchestration implementation details are hidden in private helpers.

4.5 Open/Closed Principle

The pricing system is closed for modification but open for extension. Adding a new vehicle type (e.g. MOTORCYCLE) requires only a new class extending BasePricingStrategy and a new entry in PricingStrategyFactory. No existing strategy class or service method is touched.

4.6 Dependency Inversion Principle

Both ReservationController and ReservationServiceImpl depend on abstractions — interfaces — not concrete implementations. Spring injects concrete classes at runtime. This makes each layer independently testable and replaceable.

---

5. Key Design Decisions

5.1 Backend Owns All Pricing Logic

The most important architectural decision: all pricing calculations live exclusively in the Java strategy classes. The frontend is a pure renderer.

Problem	Frontend was duplicating all pricing rules — $20/day, $15/day, 10% cleaning fee, $0.50/mile, 3-year surcharge threshold. Any rate change required updating both Java and JSX.
Decision	Backend sends a breakdown array in every response. Frontend only renders what it receives.
Impact	Rate change requires only one Java file change. Frontend never has stale values. No risk of breakdown lines not matching the total.

5.2 getOptions Returns Pricing Summary and Surcharge Flags

The getOptions endpoint was extended to return pricingSummary, surchargeApplied, and surchargePercentage alongside bookingAmount. This eliminated all hardcoded rate text from the frontend results page.

Before	Frontend had hardcoded strings like '$22/day + 10% cleaning fee'
After	Backend builds these strings from its constants — getPricingSummary() on each strategy
Benefit	Rate text is always consistent with actual calculations — single source of truth

5.3 Optimistic Locking on Vehicle

The Vehicle entity has a @Version field. This prevents two concurrent requests from double-booking the same vehicle. If two transactions read the same vehicle simultaneously, the second save will throw OptimisticLockException and roll back.

5.4 Lazy Loading on Reservation.vehicle

The Vehicle association on Reservation uses FetchType.LAZY. The vehicle is only loaded from the database when accessed — not on every reservation query. The service extracts the vehicle reference to a local variable to avoid multiple proxy calls.

5.5 Switch Expressions Over if/else Chains

Category-specific validation uses a switch expression instead of if/else. Switch on an enum compiles to tableswitch bytecode — O(1) lookup. The compiler also enforces exhaustive coverage of all enum values.

---

6. Complete Decisions Summary

Frontend Performance

Technique	File	Benefit
React.memo	CarIcon, CategoryBadge, VehicleRow, PricingSummary	Skip re-render when props unchanged
useCallback	ResultsPage, BookingPage	Stable refs — memo stays effective
Lazy loading	App.jsx — all 5 pages	Smaller initial bundle
Correct keys	ResultsPage, AmountBreakdown	Fewer DOM ops
useTransition	ResultsPage.fetchOptions	UI responsive during render

Backend Performance

Technique	File	Benefit
Java record	PricingRequest, BreakdownLineDto	Immutable, no builder overhead
static final Logger	All strategy + service classes	One logger per class
List.of()	Sedan, Van, SUV breakdown	Immutable compact list
ArrayList(3)	PickupTruck breakdown	Pre-sized, no resize

Primitive constants	All strategy classes	No boxing overhead
Single strategy lookup	ReservationServiceImpl	One map lookup per operation

OOP Principles and design patterns

Principle	Where Applied	What It Achieves
Abstraction	PricingStrategy interface	Callers depend on contract, not implementation
Strategy Pattern	PricingStrategyFactory + 4 strategies	New vehicle type = new class only
Encapsulation	Vehicle.reserve/release/isAvailable	State transitions owned by entity
Encapsulation	Reservation.cancel/isActive/updateDates	State + mutation owned by entity
SRP	Private helpers in ReservationServiceImpl	Each method has one job
OCP	BasePricingStrategy + factory	Closed for modification, open for extension
DIP	Controller + service constructor injection	Depends on interfaces not concrete classes
Polymorphism	pricingStrategyFactory.getStrategy()	Correct strategy resolved at runtime

---

End of document.